Informe proyecto integrador

Asistente de análisis de datos con IA generativa y Spark sobre el Sistema de Alertas estudiantiles a nivel de Bogotá

Integrantes equipo	Adrian Grateron Jhonatan Smith Doncel Jorge Emmanuel Marin
Dataset	SistemadeAlertas.xlsx, hoja Data General
Aplicación	Spark + GLM de Z.ai + Gradio + Docker + Hugging Face Spaces
URL pública del Space	https://huggingface.co/spaces/Adrian0812/analista-datos-ia

1. Objetivo y alcance

El proyecto consistió en construir y desplegar un asistente de análisis de datos que convierte preguntas en lenguaje natural en código PySpark ejecutable. La solución integra procesamiento distribuido con Spark, generación de código mediante el modelo GLM de Z.ai, una interfaz web en Gradio y despliegue en Hugging Face Spaces mediante un contenedor Docker.

2. Dataset seleccionado y preparación en Spark

La fuente utilizada fue el archivo **SistemadeAlertas.xlsx**, hoja Data General, con 174.826 registros y 24 columnas originales. El rango temporal de la columna de alerta va del 17 de enero de 2025 al 31 de marzo de 2026. El conjunto incluye variables categóricas como tipo_alerta, genero, localidad_nombre, nivel_academico, jornada y características diferenciales; variables numéricas como edad_caso, edad_real, estrato, anio y semestre; y variables de fecha como fecha_alerta y fecha_nacimiento.

En el notebook y en app.py, el Excel se lee inicialmente con pandas por compatibilidad con archivos xlsx, pero el análisis se realiza sobre un DataFrame de Spark llamado df. Las columnas se renombran a formato técnico estable, se define un esquema explícito con tipos LongType, IntegerType, DateType y StringType, y se agregan las variables derivadas mes_alerta y trimestre_alerta. Esto permite que el LLM genere instrucciones PySpark consistentes y evita errores por nombres con tildes, espacios o tipos inferidos de forma ambigua.

3. Funcionamiento técnico del asistente

El flujo operativo es: pregunta del usuario → instrucción al LLM con el esquema real de df → generación de código PySpark → validación básica del código → ejecución controlada → presentación de tabla y gráfico cuando el resultado es bivariado y numérico. La instrucción del modelo restringe el uso a df, spark y F, exige asignar el resultado a la variable resultado y evita que el modelo lea o escriba archivos. Adicionalmente, la aplicación bloquea patrones de riesgo como import, open, eval, exec, subprocess, operaciones de escritura o accesos al sistema.

La interfaz Gradio permite ingresar preguntas libres y ofrece ejemplos predefinidos. La aplicación muestra tres salidas: el código PySpark generado, la tabla resultante y un gráfico automático cuando corresponde. Para el despliegue, el Dockerfile instala Java, dependencias de Python y Spark, copia app.py y SistemadeAlertas.xlsx, expone el puerto 7860 y ejecuta la aplicación. La clave de Z.ai no queda escrita en el código: se configura como secret del Space bajo el nombre ZAI_API_KEY.

4. Preguntas de ejemplo, resultados e interpretación

¿Cuántas alertas hay por tipo_alerta?



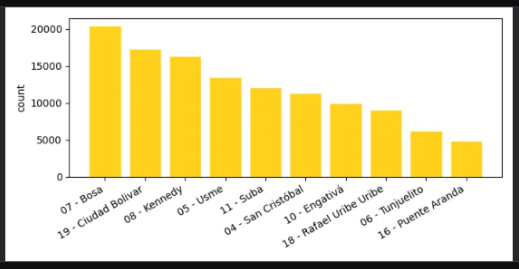
Interpretación:
La estructura de alertas está altamente concentrada en accidentalidad y abuso/violencia, categorías que reúnen cerca de 84% de los registros. Esto orienta la priorización analítica y operativa del sistema.

Muestra las 10 localidades con más alertas en 2025.

Resultado

Localidad	count
07 - Bosa	20439
19 - Ciudad Bolivar	17299
08 - Kennedy	16352
05 - Usme	13486
11 - Suba	12151
04 - San Cristóbal	11333

Gráfico (si aplica)



Usar vía API · Construido con Gradio · Configuración

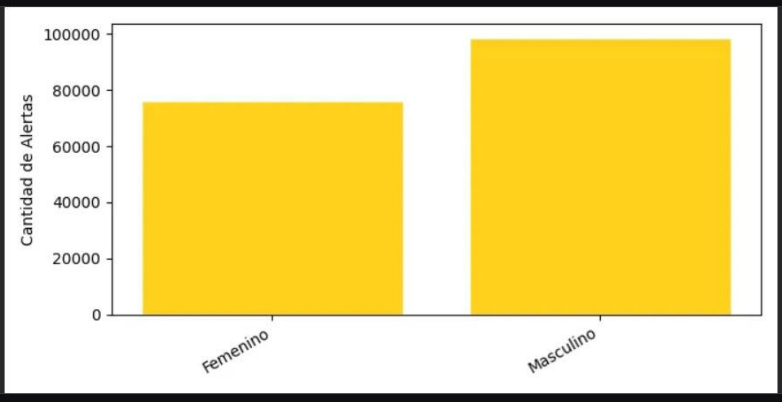
Interpretacion:
La distribución territorial muestra mayor carga de alertas en localidades del suroccidente y sur de Bogotá. Bosa, Ciudad Bolívar y Kennedy concentran 38,6% de los registros de 2025.

¿Cuál es la distribución de alertas por genero?

Resultado

Genero	Cantidad de Alertas
Femenino	76088
Masculino	98738

Gráfico (si aplica)



Interpretación:

La diferencia por género sugiere una mayor participación de registros asociados a hombres. Este resultado debe cruzarse con tipo de alerta, edad y localidad para evitar conclusiones agregadas simplificadas.

5. Conclusiones

El proyecto cumple el propósito de integrar datos reales, Spark, IA generativa y despliegue en la nube en una aplicación funcional de análisis asistido. La solución no entrena un modelo predictivo; su valor está en habilitar análisis exploratorio y descriptivo mediante text-to-code, reduciendo la barrera técnica para consultar una base amplia y con múltiples dimensiones. El uso de un esquema explícito y de nombres de columnas normalizados mejora la estabilidad de las respuestas del LLM y facilita la trazabilidad del código generado.

Como resultado analítico inicial, el Sistema de Alertas evidencia concentración por tipo de alerta y por territorio, lo que permite formular preguntas de priorización, seguimiento temporal y segmentación poblacional. Para una versión productiva, se recomienda complementar la solución con autenticación, bitácora de consultas, validaciones semánticas más estrictas, control de costos de API, monitoreo de errores del LLM y actualización periódica del archivo fuente.